ASSIGNMENT 9

Advanced SQL Queries Using Joins and Subqueries

Food Delivery Management System

Multi-table reports, summary statistics and related records

Student Name	Jeevitha S
Subject	Database Management Systems
Project	Food Delivery Management System
Database Platform	MySQL 8.0

Prepared with reference to the finalized eight-table project database design.

1. Objective

This assignment develops multi-table reports using INNER JOIN, LEFT JOIN, aggregate functions and subqueries. The reports answer practical questions about orders, customers, restaurants, payments and delivery partners.

SQL dialect and naming note: The scripts use MySQL 8.0. The project design calls its order table *Order*, but ORDER is a SQL keyword. To keep every command executable without quoting a reserved word, this report names that table **Food_Order**. It represents the same Order entity in the finalized design. All other table names and relationships are retained.

1. Order details with customer and restaurant (INNER JOIN)

```
SELECT o.Order_ID, c.Name AS Customer_Name,
       r.Restaurant_Name, o.Total_Amount, o.Order_Status
FROM Food_Order o
INNER JOIN Customer c ON o.Customer_ID = c.Customer_ID
INNER JOIN Restaurant r ON o.Restaurant_ID = r.Restaurant_ID
ORDER BY o.Order_ID;
```

Expected output: Combines order, customer and restaurant data into one readable report.

Order_ID	Customer_Name	Restaurant_Name	Total_Amount	Order_Status
5001	Aarav Kumar	Spice Garden	360.00	Delivered
5002	Diya Sharma	Green Bowl Cafe	280.00	Delivered
5003	Rahul Nair	Pizza Corner	350.00	Out for Delivery
5004	Aarav Kumar	Green Bowl Cafe	300.00	Preparing
5005	Meera Iyer	Spice Garden	240.00	Cancelled
5006	Arjun Rao	Pizza Corner	500.00	Delivered

2. Itemized order report (multiple joins)

```
SELECT o.Order_ID, c.Name AS Customer_Name,
       mi.Item_Name, oi.Quantity, oi.Unit_Price,
       oi.Quantity * oi.Unit_Price AS Line_Total
FROM Food_Order o
JOIN Customer c ON o.Customer_ID = c.Customer_ID
JOIN Order_Item oi ON o.Order_ID = oi.Order_ID
JOIN Menu_Item mi ON oi.Item_ID = mi.Item_ID
ORDER BY o.Order_ID, oi.Order_Item_ID;
```

Expected output: Shows the ordered menu items, quantities and line totals. Output is shortened to the first seven rows; the query returns all 11 order-item rows.

Order_ID	Customer_Name	Item_Name	Quantity	Unit_Price	Line_Total
5001	Aarav Kumar	Paneer Biryani	1	180.00	180.00
5001	Aarav Kumar	Veg Fried Rice	1	140.00	140.00
5001	Aarav Kumar	Gulab Jamun	1	40.00	40.00
5002	Diya Sharma	Veg Salad Bowl	1	150.00	150.00
5002	Diya Sharma	Millet Khichdi	1	130.00	130.00
5003	Rahul Nair	Margherita Pizza	1	250.00	250.00
5003	Rahul Nair	Garlic Bread	1	100.00	100.00

3. Payment report

```
SELECT o.Order_ID, c.Name AS Customer_Name,
       p.Payment_Method, p.Payment_Status, p.Amount
FROM Food_Order o
JOIN Customer c ON o.Customer_ID = c.Customer_ID
LEFT JOIN Payment p ON o.Order_ID = p.Order_ID
ORDER BY o.Order_ID;
```

Expected output: LEFT JOIN keeps the order in the report even if a payment row is not yet present.

Order_ID	Customer_Name	Payment_Method	Payment_Status	Amount
5001	Aarav Kumar	UPI	Successful	360.00
5002	Diya Sharma	Card	Successful	280.00
5003	Rahul Nair	UPI	Successful	350.00
5004	Aarav Kumar	Cash	Pending	300.00
5005	Meera Iyer	UPI	Failed	240.00
5006	Arjun Rao	Card	Successful	500.00

4. Delivery partner activity

```
SELECT dp.Partner_ID, dp.Name AS Partner_Name,
       COUNT(d.Delivery_ID) AS Assigned_Deliveries
FROM Delivery_Partner dp
LEFT JOIN Delivery d ON dp.Partner_ID = d.Partner_ID
GROUP BY dp.Partner_ID, dp.Name
ORDER BY dp.Partner_ID;
```

Expected output: LEFT JOIN includes partners even when they have no delivery assignments.

Partner_ID	Partner_Name	Assigned_Deliveries
801	Kiran Das	2
802	Nisha Patel	1
803	Sanjay B	1

5. Subquery: orders above the average order value

```
SELECT Order_ID, Total_Amount
FROM Food_Order
WHERE Total_Amount >
      (SELECT AVG(Total_Amount) FROM Food_Order)
ORDER BY Total_Amount DESC;
```

Expected output: The average order value is 338.33. The query returns orders above that average.

Order_ID	Total_Amount
5006	500.00
5001	360.00
5003	350.00

6. Subquery: customers who placed at least two orders

```
SELECT Customer_ID, Name
FROM Customer
WHERE Customer_ID IN (
    SELECT Customer_ID
    FROM Food_Order
    GROUP BY Customer_ID
    HAVING COUNT(*) >= 2
);
```

Expected output: Finds customers whose ID appears in two or more order records.

Customer_ID	Name
1	Aarav Kumar

7. Restaurant-wise order summary

```
SELECT r.Restaurant_Name,
       COUNT(o.Order_ID) AS Number_of_Orders,
```

```
       SUM(o.Total_Amount) AS Total_Order_Value,
       ROUND(AVG(o.Total_Amount),2) AS Average_Order_Value
FROM Restaurant r
LEFT JOIN Food_Order o ON r.Restaurant_ID = o.Restaurant_ID
GROUP BY r.Restaurant_ID, r.Restaurant_Name
ORDER BY r.Restaurant_ID;
```

Expected output: Provides order count, total value and average value for each restaurant. Totals include the cancelled order in the sample data.

Restaurant_Name	Number_of_Orders	Total_Order_Value	Average_Order_Value
Spice Garden	2	600.00	300.00
Green Bowl Cafe	2	580.00	290.00
Pizza Corner	2	850.00	425.00

8. Orders with delivery details

```
SELECT o.Order_ID, dp.Name AS Delivery_Partner,
       d.Delivery_Status, d.Pickup_Time, d.Delivery_Time
FROM Food_Order o
JOIN Delivery d ON o.Order_ID = d.Order_ID
JOIN Delivery_Partner dp ON d.Partner_ID = dp.Partner_ID
ORDER BY o.Order_ID;
```

Expected output: Connects order records to the assigned delivery partner and delivery status.

Order_ID	Delivery_Partner	Delivery_Status
5001	Kiran Das	Delivered
5002	Nisha Patel	Delivered
5003	Sanjay B	Picked Up
5006	Kiran Das	Delivered

9. Conclusion

The advanced queries combine related tables to produce useful operational reports. Joins connect related records, LEFT JOIN preserves unmatched parent records, and subqueries compare records against calculated or grouped results. The expected outputs are based on the sample dataset inserted in Assignment 7.